Standard features provided by Unison RTOS and compatibility with Compiler's standard C/C++ libraries.

ISO C99 integration and compatibility

1. Features Implemented in Unison RTOS v5.2.2. libraries

Table 1. Standard C features implemented in Unison RTOS.

Library name	Function names and data types	Comments
iolib library	accept, bind, chmod, close, connect, creat, fstat, getpeername, getsockname, getsockopt, ioctl, link, listen, lseek, mkdir, mkfs, mount, open, read, recv, recvfrom, rmdir, select, send, sendto, setsockopt, shutdown, socket, stat, sync, umount, unlink, write, rename, nfsmount	Fully implemented
	renameat	Subroutines with minimal functionality to allow your programs to link with the subroutines in libc.a
stdio library	FILE, size_t, fpos_t, _stdin, _stdout, _stderr, stdin_init, get_stdin_ptr, stdin_close, stdout_init, et_stdout_ptr, stdout_close, stderr_init, get_stderr_ptr, stderr_close, fopen, freopen, fdopen, fclose, fgetc, fgets, fread, fscanf, getc, getchar, gets, scanf, sscanf, vscanf, vsscanf, vfscanf, ungetc, fprintf, fputc, fputs, fwrite, printf, putc, putchar, puts, sprint, vsprintf, vprintf, vfprintf, perprintf, noperprintf, clearer, feof, ferror, fflush, fileno, fseek, ftell, rewind, setbuf, remove, rename, snprintf, vsnprintf, perror, setbuf, setvbuf, fseeko, ftello, fgetpos, fsetpos, getdelim, getline, getc_unlocked, putc_unlocked, putchar_unlocked, getchar_unlocked, dprintf, vdprintf.	Fully implemented
	ctermid, flockfile, ftrylockfile, funlockfile, tmpnam, tempnam, popen, pclose, fmemopen, open_memstream, tmpfile,	Subroutines with minimal functionality to allow your programs to link with the subroutines in libc.a

Kernel library	1. Pthreads (POSIX.1c) pthread_create() pthread_exit() pthread_self() pthread_equal() pthread_join() pthread_detach() pthread_setschedparam() pthread_getschedparam() pthread_attr_init() pthread_attr_destroy() pthread_attr_setstackaddr() pthread_attr_getstackaddr() pthread_attr_setstacksize() pthread_attr_getstacksize() pthread_attr_setschedparam() pthread_attr_getschedparam() pthread_attr_getdetachstate() pthread_attr_setdetachstate() pthread_stackinfo() pthread_setprio() pthread_getprio() sched_get_priority_max() sched_get_priority_min() sched_yield()	Fully implemented
	2. Pthread Cancellation (POSIX.1c) pthread_cleanup_pop() pthread_cleanup_push() pthread_cancel() pthread_setcanceltype() pthread_setcancelstate() pthread_testcancel()	Fully implemented
	3. Mutex (POSIX.1c) pthread_mutex_init() pthread_mutex_destroy() pthread_mutex_lock() pthread_mutex_trylock() pthread_mutex_unlock()	Fully implemented
	4. Semaphore (POSIX.1b) sem_open() sem_close() sem_unlink() sem_init() sem_destroy() sem_wait() sem_trywait() sem_timedwait() sem_post() sem_getvalue()	Fully implemented

	5. Message Queue (POSIX.1b) mq_open() mq_close() mq_unlink() mq_send() mq_receive() mq_notify() mq_setattr() mq_getattr() mq_timedreceive() mq_timedsend()	Fully implemented
	6. Conditional Variables (POSIX.1c) pthread_cond_init() pthread_cond_destroy() pthread_cond_wait() pthread_cond_timedwait() pthread_cond_signal() pthread_cond_broadcast() pthread_condattr_init() pthread_condattr_destroy()	Fully implemented
	7. Barrier (POSIX.1c) pthread_barrier_init() pthread_barrier_destroy() pthread_barrier_wait()	Fully implemented
	8. Timers (POSIX.1b) timer_create() timer_delete() timer_settime() timer_gettime() timer_getoverrun() timer_tick() nanosleep()	Fully implemented
	9. Clocks (POSIX.1) time() , sleep() , clock_settime() , clock_gettime(), clock_getres() , clock_init()	Fully implemented
	10. Memory Allocation 10.1 POSIX.1 malloc() , free() 10.2 Variable Length (Pools) pool_create() pool_destroy() pool_alloc() pool_free() 10.3 Fixed Length (Partitions) pt_create(), pt_destroy(), pt_getblock() pt_freeblock()	Fully implemented
	11. Rendezvous mr_init(), mr_send(), mr_receive(), mr_reply(), mr_sigrecv() , mr_sigpost()	Fully implemented

	12. Interrupts i_set_interrupt(), i_disable(), i_restore()	Fully implemented
	13. Directory Services dir_register(), dir_deregister(), dir_lookup(), dir_lookup_string()	Fully implemented
	14. Miscellaneous checkIstack() NanoStart() or DSPexec_Start() Nano_lock (), Nano_unlock (), _isrStackFill , kfatal(), kalloc() , kfree() pthreadStackFill, thread_numb() , thread_utilization_start() thread_utilization_stop() xprintf() , xputs() , xputchar()	Fully implemented
Unison_libc	1. Miscellaneous abort(), assert(), realloc(), strdup(), strftime(), strcasecmp(), strncasecmp()	Fully implemented
	2. Implementation of ISO C99 <wchar.h > wint_t; mbstate_t; btowc; fgetwc; fgetws; fputwc; fputws; fwide; fwprintf; fwscanf; getwc; getwchar; mbrlen; mbrtowc; mbsinit; mbsnrtowcs; mbsrtowcs; open_wmemstream; putwc; putwchar; swprintf; swscanf; ungetwc; vfwprintf; vfwscanf; vswprintf; vswscanf; vwprintf; vwscanf; wcpcpy; wcpncpy; wcrtomb; wcscasecmp; wcscat; wcschr; wcscmp; wcscoll; wcscpy; wcscspn; wcsdup; wcsftime; wcslen; wcsncasecmp; wcsncat; wcsncmp; wcsncpy; wcsnlen; wcsnrtombs; wcspbrk; wcsrchr; wcsrtombs; wcsspn; wcsstr; wcstod; wcstof; wcstok; wcstol; wcstold; wcstoll; wcstoul; wcstoull; wcswidth; wcsxfrm; wctob; wcwidth; wmemchr; wmemcmp; wmemcpy; wmemmove; wmemset; wprintf; wscanf;	Subroutines with minimal functionality to allow your programs to link with the subroutines in libc.a
	3.Implementation of ISO C99 <wctype.h > wctype_t; wctrans_t; iswalnum; swalpha; iswblank; iswcntrl; iswctype; iswdigit; iswgraph; iswlower; iswprint; iswpunct; iswspace; iswupper; iswxdigit; towctrans; towlower; towupper; wctrans; wctype;	Subroutines with minimal functionality to allow your programs to link with the subroutines in libc.a
	4. Some general system functions fork, kill, isatty, getpid, execve, _exit, environ, sbrk, times, wait, exit.	Subroutines with minimal functionality to allow your programs to link with the subroutines in libc.a

2. Integration with ISO C99

Unison RTOS redefining some of functions, described in ISO C99 Standard. Table 2 specifying relationship between ISO C99 Standard features and Unison RTOS.

Table 2. ISO C99 Standard features and Unison RTOS.

Standard C library features	Standard header files	Comments
Standard ISO C99 features according to definitions in header files	<assert.h> <stddef.h> <stdint.h> <stdio.h> <time.h> <wchar.h> <wctype.h>	Unison RTOS provides header files for this group and implementations for functions, defined in header files. For more details on implemented functions, see Unison libraries description in Table 1 above.
	<errno.h> <signal.h> <stdlib.h> <string.h> <inttypes.h> <limits.h> <locale.h>	Unison RTOS provides header files for these group but not providing implementation of functions. Functions, defined in these header files, could be used from Compiler libc.a/libm.a ***Note!*** ***Some functions from <stdlib>, like:*** ***abort(), exit(), realloc(), free, malloc – implemented in Unison libraries (see above)***
	<complex.h> <ctype.h> <fenv.h> <float.h> <iso646.h> <math.h> <setjmp.h> <stdarg.h> <stdbool.h> <tgmath.h>	Headers and functions defined in these files group could be used directly from Compiler environment

ISO C++11 integration and compatibility

The elements of the C language library are also included as a subset of the C++ Standard library. Unison RTOS providing only integration layer for C libraries to C++. This is done by sets if header files, which could be found under Unison RTOS installation directory in folder: ***\include\cxx***

The C++ library includes the same definitions as the C language library organized in the same structure of header files, with the following differences:

- Each header file has the same name as the C language version but with a "c" prefix and no extension. For example, the C++ equivalent for the C language header file <stdlib.h> is <cstdlib>.
- Every element of the library is defined within the std namespace.

Such strategy supports use all C features, implemented in Unison RTOS, with C++ compiler. Table 3 specifying relationship between C++ Standard features and Unison RTOS C libraries.

In addition to C libraries integration layer, Unison RTOS redefines operators: ***new, delete***

Other features of C++ not redefined by Unison RTOS can be used directly from Compiler environment.

Table 3. C library integration with C++

Standard C++ library features	Standard header files	Comments
Standard C++11 headers for C99	<cassert> (assert.h) <cerrno> (errno.h) <climits> (limits.h) <clocale> (locale.h) <cstdint> (stdint.h) <cstdio> (stdio.h) <ctime> (time.h) <cwchar> (wchar.h) <cwctype> (wctype.h) <new>	Unison RTOS provides header files for this group and implementations for functions, defined in header files. For more details on implemented functions, see Unison libraries description in Table 1 above.
	<csignal> (signal.h) <cstdbool> (stdbool.h) <cstdarg> (stdarg.h) <cstddef> (stddef.h) <cstdlib> (stdlib.h) <cstring> (string.h)	Unison RTOS provides header files but not providing implementations of functions. Functions, defined in these header files, could be used from Compiler libraries. ***Note!*** ***Some functions from <stdlib>, like: abort(), exit(), realloc(), free, malloc – implemented in Unison libraries (see above)***
	<cctype> (ctype.h) <cfloat> (float.h) <ciso646> (iso646.h) <cmath> (math.h). <csetjmp> (setjmp.h) <cuchar> (uchar.h)	Headers and functions defined in these files could be used directly from Compiler environment